



Supervised Learning

CS 541 Lecture 2

Zining Zhu



The supervised learning recipe

Data · Hypothesis class · Loss

What is supervised learning?

Learn a function from labeled examples so it generalizes to new, unseen inputs.

- **Data:** pairs (x, y) of inputs and target labels
- **Hypothesis class:** the family of functions we search over (e.g. linear models, trees, neural nets)
- **Loss function:** measures how wrong a prediction is
- **Training:** search the hypothesis class for parameters that minimize loss



The recipe, written formally

- Data: a training set $\{(x_1, y_1), \dots, (x_n, y_n)\}$, assumed drawn from some unknown distribution
- Hypothesis class: a family of functions $h(x; \theta)$ indexed by parameters θ
- Loss: a function $L(h(x; \theta), y)$ scoring one prediction against its true label
- Training objective: choose θ to minimize the **average** loss over the training set
- Everything in this course — logistic regression, neural nets, even reinforcement learning's policies — instantiates this same recipe with different h , L , and search strategy



A running example: will a student pass the quiz?

We will reuse this toy problem through the whole lecture.

- x = hours studied (a single feature, for now)
- $y = 1$ if the student passed, 0 if not
- Ten students, x ranging from 0.5 to 9.5 hours, mostly passing above ~5 hours
- Question for today: what hypothesis class, what loss, and how do we search for good parameters?

Small and low-dimensional on purpose — so every formula in this lecture can be checked by hand.



What goes into x: feature engineering

Real inputs are rarely a single clean number.

- **Raw numeric features:** hours studied, house square footage, pixel intensities
- **Categorical features:** encoded as **one-hot** vectors (e.g. major: CS/EE/ME → three 0/1 columns)
- **Derived features:** ratios, counts, interaction terms built from raw inputs
- Feature choices matter as much as the model — a good feature can make a simple model work well
- We revisit feature **scaling** later today, once we talk about optimizers



Hypothesis classes: some examples

The hypothesis class is a modeling choice, made before you see any loss curve.

- **Linear models:** $h(x) = w \cdot x + b$ — few parameters, easy to fit, limited shapes
- **Decision trees:** axis-aligned splits — easy to interpret, can overfit if grown deep
- **Neural networks:** compositions of linear layers and nonlinearities — flexible, data-hungry
- Today's lecture focuses on linear and logistic models; trees and neural nets return throughout the course



Regression vs. classification

Two basic task types, distinguished by what **y** looks like.

- **Regression:** y is continuous (e.g. house price, temperature)
 - Typical loss: squared error
- **Classification:** y is categorical (e.g. spam/not-spam, digit 0-9)
 - Typical loss: cross-entropy
- Same recipe (data, hypothesis, loss, training) underlies both



Regression, a few more examples

- Predicting a house's sale price from square footage, bedrooms, location
- Predicting tomorrow's high temperature from today's weather readings
- Predicting a patient's recovery time from vitals and treatment history
- Predicting a stock's next-day return from recent price history (notoriously hard — noise dominates signal)



Classification, a few more examples

- **Binary:** spam vs. not-spam, fraud vs. legitimate transaction, malignant vs. benign tumor
- **Multi-class:** digit recognition (0-9), part-of-speech tagging, image category
- Some problems that look like regression are better framed as classification (e.g. “will it rain” is binary, not “how many mm of rain”)



Why we split the data

A model that only memorizes the training set is useless.

- **Train set:** fit the model's parameters
- **Validation set:** tune hyperparameters (learning rate, regularization strength, model complexity)
- **Test set:** report final performance, touched **once**
- Tuning on the test set leaks information and overstates how well you'll do on new data

Never let the test set influence any decision before the final report.



A cautionary tale: test-set leakage

- A common mistake: try ten models, keep the one with the best **test** accuracy
- That test accuracy is now optimistic — you effectively tuned a hyperparameter (which model) on the test set
- A subtler version: normalizing features using statistics computed over the **whole** dataset before splitting
- Rule of thumb: any computation that “looks at” labels or full-dataset statistics must happen only on the training (or training+validation) portion



k-fold cross-validation

A more data-efficient way to estimate validation performance.

- Split training data into **k** folds
- Train on $k-1$ folds, validate on the held-out fold; repeat k times
- Average the k validation scores
- Useful when the dataset is small and a single train/val split is noisy



Choosing k , and its computational cost

- Common choices: $k = 5$ or $k = 10$
- Larger k : less bias in the estimate (each training fold is closer to the full training set), but k times the training cost
- **Leave-one-out CV** is the extreme case, $k = n$: maximally data-efficient, but expensive and high-variance across folds for some models
- In practice: $k = 5$ or 10 is the default unless the dataset is tiny



Stratified splits and class imbalance

- **Stratified split:** preserve the same class proportions in every fold or split
- Matters most when one class is rare (e.g. 2% fraud, 98% legitimate)
- A random split can otherwise put almost no positive examples in a validation fold, making the validation score meaningless
- scikit-learn's `StratifiedKFold` (and similar tools) do this automatically — worth knowing it exists even before A1



Evaluating regression models

Once trained, how do we score a regression model's predictions?

- **MSE (mean squared error)**: average of $(\hat{y} - y)^2$ — penalizes large errors heavily
- **MAE (mean absolute error)**: average of $|\hat{y} - y|$ — more robust to outliers than MSE
- **R² (coefficient of determination)**: fraction of the variance in y explained by the model, 1.0 is a perfect fit, 0.0 is no better than predicting the mean
- Report more than one metric — MSE alone hides **how** a model is wrong



Evaluating classifiers: the confusion matrix

Accuracy alone can be dangerously misleading.

- A **confusion matrix** cross-tabulates predicted vs. true class: true positive (TP), false positive (FP), true negative (TN), false negative (FN)
- Example: a fraud detector that always predicts “not fraud” on a 2%-fraud dataset gets 98% accuracy while catching zero fraud
- The confusion matrix exposes exactly this failure; a single accuracy number does not



Precision and recall

- **Precision** = $TP / (TP + FP)$: of the examples we flagged positive, how many really were?
- **Recall** = $TP / (TP + FN)$: of the examples that really were positive, how many did we catch?
- High precision, low recall: cautious model, misses real cases
- Low precision, high recall: aggressive model, many false alarms
- The right balance depends on the cost of each error type (missing a fraud vs. annoying a legitimate customer)



The precision-recall tradeoff and F1

- Moving the classification threshold trades precision against recall
- **F1 score** = harmonic mean of precision and recall, a single number balancing both
- Useful when you need one metric to compare models, but always inspect precision and recall separately too — F1 can hide an extreme imbalance between them



ROC curves and AUC

- **ROC curve:** true positive rate vs. false positive rate, swept across every possible threshold
- **AUC** (area under the ROC curve): a threshold-independent summary; 1.0 is perfect, 0.5 is random guessing
- Useful for comparing models before committing to a specific operating threshold
- Complements, but does not replace, precision/recall — especially under heavy class imbalance, precision-recall curves are often more informative than ROC



Choosing a decision threshold

- A classifier outputs a **probability**; the threshold (often 0.5 by default) turns it into a decision
- Lowering the threshold: catch more positives (higher recall), accept more false alarms (lower precision)
- Choose the threshold using the validation set and the real-world cost of each error type
- This choice is a hyperparameter like any other — never tune it on the test set



Formalizing generalization: two kinds of risk

A preview of language we return to in the course's final lecture.

- **Empirical risk:** average loss on the training set — what we actually optimize
- **True (population) risk:** expected loss over the **entire** data distribution — what we actually care about
- Low empirical risk does not guarantee low true risk — that gap **is** overfitting, stated more formally
- We come back to this with real guarantees (PAC learning, generalization bounds) in the last lecture of the semester



Check your understanding: the recipe

- Can you name the four ingredients of the supervised learning recipe?
- Why is tuning hyperparameters on the test set a mistake?
- Given a 1%-positive-class dataset, why might 99% accuracy be a bad sign, not a good one?
- What's the difference between what k-fold CV estimates and what a single train/val split estimates?



Bias-variance tradeoff

Underfitting · Overfitting · Regularization

Underfitting and overfitting

Two failure modes of a learned model.

- **Underfitting:** model too simple, high error on train **and** test
- **Overfitting:** model too complex, low error on train, high error on test
- Goal: the sweet spot that generalizes



A worked example: fitting polynomials

- Toy dataset: 12 points sampled from $y = \sin(x)$ plus a little noise
- Fit polynomials of degree 1, 3, and 10 by least squares to the same 12 points
- Degree 1 (a line): cannot bend to follow $\sin(x)$ at all
- Degree 3: bends roughly the right amount
- Degree 10: has enough free parameters to pass through (almost) every training point exactly



Reading the fit at each degree

Degree	Train error	Validation error	Verdict
1	high	high	underfits
3	low	low	good fit
10	≈ 0	high	overfits

- Degree 10's near-zero training error is a warning sign, not a triumph
- The gap between train and validation error is the tell — not the train error alone



Train vs. validation error as complexity grows

- Plot both errors against polynomial degree, 1 through 10
- Train error decreases monotonically — more capacity can only fit the training set better (or equally well)
- Validation error decreases, hits a minimum, then rises again
- The minimum of the validation curve is the complexity we actually want — not the minimum of the train curve



The bias-variance decomposition

Test error decomposes into three additive pieces:

- **Expected test error** = $\text{Bias}^2 + \text{Variance} + \text{Irreducible noise}$
- This holds for squared-error loss, and the intuition transfers to other losses
- Each term has a distinct cause, and a distinct fix



Deriving the decomposition: setup

- Model the data as $y = f(x) + \epsilon$, noise ϵ with mean 0 and variance σ^2
- Train on a random training set D ; the fitted function $\hat{h}_D(x)$ is itself a random variable, since D is random
- Define $\bar{h}(x) = \mathbb{E}_D[\hat{h}_D(x)]$: the average prediction if you refit on many different training sets of the same size
- We expand $\mathbb{E}[(y - \hat{h}_D(x))^2]$ around this average to see where each term comes from



Deriving the decomposition: the expansion

- Insert and subtract $\bar{h}(x)$ to split the error into three named pieces:

$$y - \hat{h}_D(x) = \underbrace{f(x) - \bar{h}(x)}_A + \underbrace{\bar{h}(x) - \hat{h}_D(x)}_B + \underbrace{\epsilon}_C$$

- Squaring gives six terms: $(A + B + C)^2 = A^2 + B^2 + C^2 + 2AB + 2AC + 2BC$
- The three squares are exactly what we are after: A^2 is Bias², $\mathbb{E}_D[B^2]$ is Variance, $\mathbb{E}[C^2]$ is σ^2
- The three **cross terms** $2AB, 2AC, 2BC$ all have expectation 0, which is why the decomposition comes out additive with no leftovers

Why the cross terms vanish

- $A = f(x) - \bar{h}(x)$ is not random: f is fixed and $\bar{h}$ is already averaged over training sets, so A pulls straight out of any expectation
- $\mathbb{E}[2AB] = 2A \mathbb{E}_D[\bar{h}(x) - \hat{h}_D(x)] = 0$, since $\mathbb{E}_D[\hat{h}_D(x)] = \bar{h}(x)$ is the **definition** of $\bar{h}$
- $\mathbb{E}[2AC] = 2A \mathbb{E}[\epsilon] = 0$, since the noise has mean 0
- $\mathbb{E}[2BC] = 2 \mathbb{E}_D[B] \cdot \mathbb{E}[\epsilon] = 0$: the test point's ϵ is independent of D , so the product splits, and both factors are 0 already
- That last independence is what breaks on a **training** point, where $\hat{h}_D$ has already fitted that ϵ ; this is why training error reads optimistically low



Deriving the decomposition: final form

- $\mathbb{E}[(y - \hat{h}_D(x))^2] = (f(x) - \bar{h}(x))^2 + \mathbb{E}_D[(\bar{h}(x) - \hat{h}_D(x))^2] + \sigma^2$
- First term = **Bias**²: the fixed gap between the truth and the model's **average** fit
- Second term = **Variance**: how much a single fit wobbles around that average, purely from which training set you drew
- Third term = **noise** σ^2 : never involved $\hat{h}_D$, so no model choice touches it



Unpacking bias

- **Bias:** the error from the hypothesis class itself being unable to represent the true function
- A linear model has high bias on data generated by a sine wave, no matter how much data you feed it
- Fix: use a richer hypothesis class (higher-degree polynomial, deeper network, more features)
- Bias does **not** shrink with more training data alone



Unpacking variance

- **Variance:** how much the fitted model would change if you re-sampled the training set
- A degree-10 polynomial fit to 12 points swings wildly with small changes to those 12 points
- Fix: more training data, a simpler hypothesis class, or regularization
- Variance **does** shrink with more training data



Irreducible noise

- **Irreducible noise:** error present in the data itself, no model can remove it
- Example: two students who studied the same 4 hours, one passes, one doesn't — some outcome variance is just not explained by the feature we recorded
- Sets a floor on achievable test error, however good the model or however much data you collect
- Estimating this floor (when possible) tells you when further modeling effort has hit diminishing returns



Simple vs. complex models

- Simple models: high bias, low variance — consistently wrong in the same way
- Complex models: low bias, high variance — right on average, but unstable from one training set to the next
- No model choice escapes this tradeoff entirely; the practical question is **where** on the curve to sit
- Ensembling, briefly: averaging many high-variance models (e.g. bagged trees) can cancel out variance while keeping low bias



Model complexity and regularization

Regularization trades a bit of bias for a lot less variance.

- Add a penalty term to the loss that discourages large weights
- **L2 (ridge)**: penalizes squared weight magnitude, shrinks weights smoothly toward zero
- **L1 (lasso)**: penalizes absolute weight magnitude, drives some weights exactly to zero (sparsity)
- Regularization strength is a hyperparameter, tuned on the validation set



L2 regularization in detail

- Adds $\lambda \cdot \sum_j w_j^2$ to the loss, where λ controls the penalty's strength
- Larger $\lambda \rightarrow$ smaller weights $\rightarrow$ simpler effective model $\rightarrow$ higher bias, lower variance
- $\lambda = 0$ recovers plain (unregularized) training
- Geometrically: shrinks every weight toward zero, proportionally more for large weights



L1 regularization in detail

- Adds $\lambda \cdot \sum_j |w_j|$ to the loss instead
- The penalty's corner at zero pushes some weights to **exactly** zero, not just small
- Result: automatic feature selection — the model effectively ignores the zeroed-out features
- Useful when you suspect only a handful of your many features actually matter



L1 vs. L2

	L2 (ridge)	L1 (lasso)
Penalty	$\sum_j w_j^2$	$\sum_j w_j $
Effect on weights	shrinks smoothly	many exactly zero
Feature selection	no	yes, implicitly
Solution	closed-form friendly	requires iterative/subgradient methods



Choosing the regularization strength

- λ is a hyperparameter: tune it on the validation set, or with k-fold cross-validation
- A typical workflow: try λ over a log-spaced grid (e.g. 0.0001, 0.001, 0.01, 0.1, 1, 10), pick the value with the best average validation score
- Too small a λ : back to overfitting. Too large: back to underfitting (the penalty dominates the actual loss)
- This is the same “sweet spot” search as choosing polynomial degree — regularization strength is just another complexity knob



Learning curves as a diagnostic

Plot train and validation error against training set size (or training iterations).

- Both errors high, close together → **underfitting** (high bias)
- Train error low, validation error much higher → **overfitting** (high variance)
- Gap between curves narrows with more data → more data helps
- Use learning curves to decide: more data, more regularization, or a bigger model?



Reading learning curve shape: underfitting

- Both curves plateau at a **high** error, close to each other
- More training data barely moves either curve — the model has hit its ceiling
- The fix is not “collect more data”; it’s “use a more expressive hypothesis class”



Reading learning curve shape: overfitting

- Train error stays low throughout; validation error stays noticeably higher
- The gap between them is the signature of high variance
- More data, stronger regularization, or a simpler model all attack this gap directly



Reading learning curve shape: more data helps

- Both curves are still converging toward each other as training set size grows
- This is the “good problem to have” — collecting more data is likely to keep improving validation performance
- Distinguishing this from the overfitting case is exactly why we plot the **curve**, not just one point



Check your understanding: bias-variance

- Which term in the decomposition would more training data reduce: bias, variance, or both?
- Why does L1 zero out weights while L2 only shrinks them?
- Given a learning curve where both errors are high and flat, what should you try next?
- Why is a training error of exactly 0 a warning sign rather than good news?



Logistic regression

Sigmoid · Cross-entropy · Softmax

From linear to logistic regression

Linear regression predicts an unbounded real number; classification needs a probability.

- Linear regression: prediction = $w \cdot x + b$, any real value
- We want a prediction in $[0, 1]$ to represent $P(y = 1 \mid x)$
- The **sigmoid** function squashes any real number into $(0, 1)$
- Logistic regression: prediction = $\text{sigmoid}(w \cdot x + b)$



The sigmoid function

- $\sigma(z) = \frac{1}{1 + e^{-z}}$
- $\sigma(0) = 0.5$, $\sigma(z) \rightarrow 1$ as $z \rightarrow +\infty$, $\sigma(z) \rightarrow 0$ as $z \rightarrow -\infty$
- Monotonic and smooth: larger z always means higher probability, and the function has no jumps
- $z = w \cdot x + b$ is called the **logit**: the sigmoid maps the logit to a probability



Properties of the sigmoid

- Symmetric around (0, 0.5): $\sigma(-z) = 1 - \sigma(z)$
- Its derivative has an elegant closed form: $\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$
- This derivative is largest near $z = 0$ and vanishes far from 0 — the model learns fastest when it is **unsure**
- This exact derivative is what makes the gradient of logistic regression so clean, as we will see shortly



Interpreting the output as a probability

- $\hat{y} = \sigma(w \cdot x + b)$ is read as $P(y = 1 \mid x; w, b)$
- A prediction of 0.5 says “genuinely unsure”; 0.9 says “confident positive”; 0.1 says “confident negative”
- This is a real advantage over a plain linear model: the output is directly usable for ranking, thresholding, or downstream decision-making



Decision boundary

- **Decision boundary**: the set of points where predicted probability = 0.5
- That happens exactly where the logit $w \cdot x + b = 0$
- $w \cdot x + b = 0$ is a linear equation in x , so the decision boundary is a straight line (a hyperplane in higher dimensions)
- Logistic regression is a **linear classifier**: despite the nonlinear sigmoid, the boundary it draws is straight



Why squared error is a poor loss for classification

- Squared error on a sigmoid output has very flat gradients when the model is confidently **wrong**
- Concretely: if $y = 1$ but the model predicts $\hat{y} \approx 0$, $\sigma'(z) \approx 0$ there, so the gradient of squared error nearly vanishes — the model barely updates, exactly when it most needs to
- We need a loss whose gradient stays informative even when the prediction is confidently wrong



Cross-entropy loss, defined

- For a single example:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

- If $y = 1$: $L = -\log(\hat{y})$ — grows without bound as $\hat{y} \rightarrow 0$
- If $y = 0$: $L = -\log(1 - \hat{y})$ — grows without bound as $\hat{y} \rightarrow 1$
- Total loss: average this across all training examples



Cross-entropy loss, intuition

- Confidently right ($\hat{y}$ close to the true label): loss near 0
- Confidently **wrong**: loss grows very large, unboundedly
- This is exactly the behavior we wanted: strong, informative gradients when the model is wrong, regardless of how “confident” that wrong prediction is
- Minimizing cross-entropy is equivalent to maximizing the likelihood of the observed labels under the model (maximum likelihood estimation)



Deriving the gradient: setting up

We derive ∇L with respect to the weights w , one example at a time.

- $\hat{y} = \sigma(z), z = w \cdot x + b$
- $L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$
- By the chain rule: $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$
- We compute each of the three factors separately, then multiply



Deriving the gradient: the three factors

- $\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$
- $\frac{\partial \hat{y}}{\partial z} = \sigma'(z) = \hat{y}(1 - \hat{y})$ — the clean sigmoid derivative from earlier
- $\frac{\partial z}{\partial w} = x$
- Multiplying the first two factors alone causes a dramatic simplification, shown next



Deriving the gradient: the simplification

- $\left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right) \cdot \hat{y}(1-\hat{y})$ simplifies to $(\hat{y} - y)$
- The $\frac{y}{\hat{y}}$ and $\frac{1-y}{1-\hat{y}}$ terms cancel almost entirely against the $\sigma'(z)$ factor
- This cancellation is exactly **why** cross-entropy and sigmoid are paired together — the messy log and exponential terms disappear



The gradient's final, clean form

- $$\frac{\partial L}{\partial w} = (\hat{y} - y) \cdot x$$
- $$\frac{\partial L}{\partial b} = (\hat{y} - y)$$

- In words: the gradient is simply “prediction error” times “input” — as intuitive a learning rule as exists in this course
- The same clean form reappears in softmax regression and in the output layer of neural network classifiers



A worked-by-hand example: setup

- One training example: $x = 3$ (hours studied), $y = 1$ (passed)
- Current parameters: $w = 0.2$, $b = -0.5$
- Compute the logit: $z = w \cdot x + b = 0.2 \times 3 - 0.5 = 0.1$
- Compute the prediction: $\hat{y} = \sigma(0.1) \approx 0.525$



Worked example: the gradient and the update

- Gradient w.r.t. w : $(\hat{y} - y) \cdot x = (0.525 - 1) \times 3 = -1.425$
- Gradient w.r.t. b : $(\hat{y} - y) = -0.475$
- With learning rate $\eta = 0.1$: $w \leftarrow w - \eta \cdot (-1.425) = 0.2 + 0.1425 = 0.3425$
- $b \leftarrow b - \eta \cdot (-0.475) = -0.5 + 0.0475 = -0.4525$ — both moved toward predicting $y = 1$ more strongly, as expected



Multi-class extension: the softmax function

- Sigmoid handles binary classification; **softmax** generalizes to **K** classes
- Given K logits $z_1, \dots, z_K$, softmax outputs

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- Each p_i is in $(0, 1)$, and all K values sum to exactly 1 — a valid probability distribution
- Softmax turns a vector of raw scores into a probability distribution over K classes



From sigmoid to softmax

- Softmax with $K = 2$ classes reduces exactly to the sigmoid (up to a reparameterization) — sigmoid is softmax's special case
- Each class gets its own weight vector w_k ; the logit for class k is $z_k = w_k \cdot x + b_k$
- The predicted class is typically argmax over the K probabilities
- This is the same softmax used at the output of nearly every deep classification network, including the language models later in this course



Cross-entropy for multi-class

- $L = -\log(p_c)$, where c is the index of the true class
- Only the true class's predicted probability enters the loss — pushing that probability toward 1 pushes all others toward 0 automatically (since they sum to 1)
- The gradient has the same clean form as before: $\frac{\partial L}{\partial z_k} = p_k - 1[k = c]$
- “Prediction error” again: the gradient is the predicted distribution minus the one-hot true label



Why the loss is convex

- For logistic and softmax regression, the cross-entropy loss (as a function of w) is **convex**
- Convexity guarantees: no bad local minima — any point where the gradient is zero is the global minimum
- This is a big part of why logistic regression is a reliable, well-behaved baseline — gradient descent is guaranteed to find the best fit (for this hypothesis class)
- This guarantee disappears once we add hidden layers (neural networks) — a preview of why deep learning optimization is harder



Regularizing logistic regression

- Same L1/L2 penalties as before, applied to the logistic regression weights
- Prevents the model from driving weights to extreme values on separable or noisy data
- Regularization strength is again a validation-set hyperparameter



Nonlinear boundaries via feature engineering

- Logistic regression's decision boundary is linear **in the features it is given** — but the features themselves can be nonlinear
- Adding x^2 , $x \cdot x_2$, or other engineered features lets a “linear” model draw curved boundaries in the original input space
- This is the same bias-variance dial as polynomial degree from the previous section: more engineered features, more capacity, more overfitting risk
- Neural networks, covered later in the course, learn these nonlinear features automatically instead of by hand



Logistic regression as a one-layer network

- prediction = $\sigma(w \cdot x + b)$ is exactly a single artificial neuron with a sigmoid activation
- Stack more such layers, with nonlinearities in between, and you have a neural network
- Everything derived today — the loss, the clean gradient form — reappears, layer by layer, when we study language models later this semester

Logistic regression is a single-layer neural network with a sigmoid or softmax output.



Check your understanding: logistic regression

- Why is cross-entropy paired with sigmoid rather than squared error?
- What does it mean, geometrically, that logistic regression's decision boundary is “linear”?
- In the gradient $(\hat{y} - y) \cdot x$, what happens to the update when the model is already exactly right?
- Why does convexity matter for how confident we can be in a training run's result?





Optimizers

SGD · Momentum · Adam

Training as optimization

Training means minimizing a loss by moving parameters against the gradient.

- The loss $L(\theta)$ is a function of the parameters θ (weights and biases)
- The gradient $\nabla L(\theta)$ points in the direction of **steepest increase**
- Moving **against** the gradient decreases the loss, locally
- Optimizers differ in exactly how they use this one piece of information



Gradient descent, the basic loop

- **Batch gradient descent:** compute the gradient over the **entire** training set each step
- Update rule:

$$w \leftarrow w - \eta \cdot \nabla L(w)$$

- Repeat until the loss stops decreasing (or a fixed number of steps)
- Accurate gradient, but slow and memory-heavy for large datasets



Batch gradient descent: pros and cons

- Pro: the gradient is exact (given the current parameters), so each step reliably decreases the loss (for small enough η)
- Con: one step requires touching every training example — infeasible when n is in the millions
- Con: it cannot start updating until the whole dataset has been read once
- This motivates estimating the gradient from a **subset** of the data instead



Stochastic gradient descent

- **Stochastic GD (SGD):** estimate the gradient from a **single** example per step
- Noisy but fast, and the noise itself can help escape shallow local minima
- Each step is cheap, so far more steps happen per unit time than batch GD allows
- The word “stochastic” refers to this noisy, single-example gradient estimate



Why SGD's noise can help

- A noisy gradient estimate occasionally pushes the parameters **away** from a shallow local minimum or saddle point
- Batch GD, with its exact gradient, would settle into the first flat spot it finds
- This matters much more for non-convex losses (deep networks) than for logistic regression's convex loss, but the intuition is worth having early
- The noise also acts as a mild implicit regularizer in practice



Mini-batch gradient descent

- **Mini-batch GD:** estimate the gradient from a small batch (e.g. 32-256 examples)
- The practical default: balances gradient accuracy and update speed
- Also better matches how modern hardware (GPUs) computes efficiently — batched matrix multiplications
- “SGD” in most modern deep learning code actually means mini-batch GD



Comparing batch sizes

Batch size	Gradient noise	Steps per epoch	Hardware efficiency
1 (SGD)	high	most	low
32-256 (mini-batch)	moderate	many	high
Full dataset (batch GD)	none	one per epoch	depends on memory

- There is no universally “best” batch size — it trades off noise, speed, and memory
- Larger batches often need a correspondingly larger learning rate to train at the same overall pace



Learning rate

The step size η controls how far each update moves.

- Too high: loss diverges or oscillates
- Too low: training crawls, may get stuck before converging
- Often decayed over training (learning rate schedules) to fine-tune late in training



Learning rate: too high, in detail

- Each step overshoots the minimum, landing on the **other side** of the valley
- The loss can oscillate indefinitely, or diverge to increasingly large values
- Symptom to watch for: a loss curve that spikes upward, sometimes to NaN
- Fix: reduce η , often by a factor of 10, and retry



Learning rate: too low, in detail

- Each step barely moves the parameters
- Training “works” in the sense that the loss slowly decreases, but takes far longer than necessary
- Worse, on a non-convex loss, tiny steps can get trapped in a shallow local minimum that a larger step would have escaped
- Symptom to watch for: a loss curve that looks like a nearly flat line from very early on



Learning rate schedules

- Common pattern: **warm up** (start small, ramp up) then **decay** (shrink over time)
- **Step decay**: multiply η by a fixed factor (e.g. 0.1) every fixed number of epochs
- **Cosine decay**: smoothly decrease η following a cosine curve down to (near) zero
- Rationale: large steps early make fast progress; small steps late fine-tune near the minimum without overshooting



Momentum: the idea

Each optimizer fixes a specific weakness of plain SGD.

- Plain SGD can zig-zag across a loss surface that curves much more steeply in one direction than another
- **Momentum:** accumulates a running average of past gradients, smooths out this zig-zagging
- Physical analogy: a heavy ball rolling downhill, which keeps moving in a consistent direction even as the slope's direction wobbles



Momentum: the update rule

- $$v \leftarrow \beta v + (1 - \beta) \nabla L(w)$$

typically $\beta \approx 0.9$

- $$w \leftarrow w - \eta v$$

- v accumulates a decaying average of recent gradients, rather than reacting to just the latest one
- Consequence: consistent gradient directions accelerate; inconsistent (oscillating) directions cancel out



RMSProp: the idea

- **RMSProp**: divides each parameter's step by a running average of its recent gradient magnitude
- Adapts the step size **per-parameter** — parameters with historically large gradients get smaller effective steps, and vice versa
- Useful when different parameters naturally live on very different scales (common once features aren't perfectly normalized)



RMSProp: the update rule

- $$s \leftarrow \gamma s + (1 - \gamma)(\nabla L(w))^2$$

typically $\gamma \approx 0.9$ (element-wise square)

- $$w \leftarrow w - \frac{\eta \cdot \nabla L(w)}{\sqrt{s} + \epsilon}$$

ϵ a tiny constant for numerical stability

- Large recent gradients for a parameter $\rightarrow$ large $s \rightarrow$ smaller effective step for that parameter
- This adaptivity is exactly what fixes SGD's “one learning rate for every parameter” limitation

Adam: combining momentum and RMSProp

- **Adam:** combines momentum (a running average of the gradient) and RMSProp-style adaptive scaling
- Maintains both a first moment estimate (like momentum's v) and a second moment estimate (like RMSProp's s)
- Includes a bias-correction step early in training, when both running averages start at zero
- The common default optimizer today for training neural networks, including the language models covered later in this course



Adam's update rule

- $$m \leftarrow \beta_1 m + (1 - \beta_1) \nabla L(w)$$
 - first moment (mean of gradients)
- $$s \leftarrow \beta_2 s + (1 - \beta_2) (\nabla L(w))^2$$
 - second moment (mean of squared gradients)
- Bias-corrected: $\hat{m} = m / (1 - \beta_1^t)$, $\hat{s} = s / (1 - \beta_2^t)$, where t is the step count
- $$w \leftarrow w - \frac{\eta \hat{m}}{\sqrt{\hat{s}} + \epsilon}$$
 - typical defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Recap: compare different optimizers

Optimizer	Fixes	Extra state	Typical use
Batch GD	— (baseline)	none	small/convex problems
SGD	scalability	none	very large datasets, simple models
Mini-batch GD	speed + hardware fit	none	the default data pipeline
Momentum	zig-zagging	one running average	speeding up SGD
RMSProp	per-parameter scale	one running average	ill-conditioned losses
Adam	both of the above	two running averages	default for neural nets

Memory consumption: (two (old model, new model) + N extra state) times model size.



Second-order methods, briefly

- Newton's method uses the **curvature** (second derivative) of the loss, not just its slope, to take a smarter step
- In principle, converges in far fewer steps than gradient descent
- In practice: computing and inverting the curvature matrix (the Hessian) is far too expensive for models with millions of parameters
- This is exactly why deep learning relies on first-order methods (SGD, Adam) despite their comparative simplicity



Common pitfall: unnormalized features

- **Feature normalization:** scale features to similar ranges so gradients aren't dominated by one feature
- Example: “hours studied” (0-10) mixed with “family income” (0-200,000) — the loss surface becomes a long, thin valley
- A single learning rate then works poorly for both features simultaneously
- Fix: standardize each feature (subtract mean, divide by standard deviation) before training



Common pitfall: a bad learning rate choice

- The single most common reason a training run silently fails to improve
- Symptom checklist: loss increasing $\rightarrow$ too high; loss flat from step 1 $\rightarrow$ too low or a code bug
- Standard fix: sweep η over a log-spaced grid (1, 0.1, 0.01, 0.001, ...) and watch the first few hundred steps of the loss curve before committing to a long run



Common pitfall: class imbalance

- With 99% negative examples, a model can reach 99% accuracy by always predicting “negative” — recall for the rare class collapses to zero
- Symptom: high accuracy, but precision/recall on the minority class near zero (tying back to the confusion-matrix discussion earlier)
- Fixes: re-weight the loss per class, resample the training set, or choose a threshold using precision/recall rather than accuracy
- This is a data and evaluation issue, not something a fancier optimizer alone can fix



Practical tips

- **Feature normalization:** scale features to similar ranges so gradients aren't dominated by one feature
- **Batch size:** bigger batches give smoother gradients but cost more memory per step
- **Learning rate schedules:** warm up, then decay (step decay, cosine decay)
- When in doubt: normalize features, start with Adam, tune the learning rate first



Debugging a training run

- Loss not decreasing at all: check the learning rate, check for a sign error in the gradient, check that data and labels are aligned correctly
- Loss decreasing on train but not validation: this is the overfitting pattern from the bias-variance section — add regularization or data, not a different optimizer
- Loss becomes NaN: almost always the learning rate is too high, or a feature is unnormalized and produces a huge logit
- A cheap sanity check: can the model overfit a **tiny** subset (e.g. 10 examples) to near-zero training loss? If not, something in the pipeline is broken, independent of the optimizer



Check your understanding: optimizers

- Why does momentum help specifically with a loss surface that curves much more in one direction than another?
- What two things does Adam combine, and why does combining them help?
- Given a loss curve that spikes to very large values within the first few steps, what is the most likely fix?
- Why is class imbalance a problem that a better optimizer cannot solve by itself?



Recap: the two-part story of the day

Today's story had two parts: a way to **learn** a model from data, and how to **tune** it.

- Learn: hypothesis class + loss + train/val/test discipline, with confusion-matrix-based evaluation for classifiers
- Tune: bias-variance tradeoff, regularization, and the right optimizer
- Logistic regression is the simplest model that already uses all of these ideas end to end



Where these ideas resurface later in the course

- **Reward models** in RLHF (Lecture 7) are trained with the exact same supervised recipe — data, loss, optimizer — just on human preference labels
- **Contrastive pretraining** for vision-language models (Lecture 12) still bottoms out in a cross-entropy-style loss and gradient descent
- **Generalization**, informally discussed today via bias-variance, gets a formal treatment with PAC bounds in the last lecture of the semester
- The recipe you learned today does not go away as models get larger — it gets reused, layer after layer



Looking ahead

- Next week: **unsupervised learning** — learning structure without labels
- **Assignment 1** (supervised learning in Python) due September 17 at 3pm
- Bring questions about the gradient derivation or the bias-variance decomposition to office hours (Wednesdays 10-12, GS429 or Zoom)

Everything today assumed labeled data. Next week asks what we can learn without it.





THANK YOU

Stevens Institute of Technology
1 Castle Point Terrace, Hoboken, NJ 07030